

Practical 7: Multistage Graph Problem

Aim

Write a program to implement the Multistage Graph problem using Dynamic Programming and find the minimum cost from the source vertex to the destination vertex, using both the forward and backward approach.

Theory

Multistage Graph

A multistage graph is a directed, weighted graph in which the vertex set is partitioned into k disjoint subsets (stages) $V_1, V_2, \dots, V_k$, such that every edge (u, v) goes from a vertex in stage V_i to a vertex in stage V_{i+1} . Stage V_1 contains exactly one vertex called the source, and stage V_k contains exactly one vertex called the sink (destination). The goal is to find the minimum cost path from the source to the sink.

Dynamic Programming Formulation

Because the graph is staged, every path from source to destination passes through exactly one vertex from each stage, so the problem exhibits both optimal substructure and overlapping subproblems. Let $\text{cost}(i)$ denote the minimum cost from vertex i to the destination. The forward recurrence is $\text{cost}(i) = \min \text{ over all edges } (i, j) \text{ of } \{ c(i, j) + \text{cost}(j) \}$, computed backward from the destination toward the source. Equivalently, in the backward approach, let $\text{bcost}(j)$ denote the minimum cost from the source to vertex j : $\text{bcost}(j) = \min \text{ over all edges } (i, j) \text{ of } \{ \text{bcost}(i) + c(i, j) \}$, computed forward from the source toward the destination.

Why Dynamic Programming Works Here

Both formulations recompute the optimal cost of a vertex only once and reuse it for every vertex that depends on it, which is the essence of dynamic programming. The staged structure guarantees an acyclic, topologically ordered graph, so each vertex's optimal cost depends only on vertices already solved in the previous stage, avoiding recomputation and ensuring the recurrence terminates.

Complexity

If the graph has n vertices and e edges, both the forward and backward dynamic programming approaches run in $O(n + e)$ time, since every vertex and every edge is examined exactly once. This is a significant improvement over a naive path-enumeration approach, which would be exponential in the number of stages.

Algorithm

Forward Approach:

1. Divide the graph into k stages, with stage 1 as the source and stage k as the destination.
2. Set $\text{cost}(\text{destination}) = 0$.
3. For each vertex i, moving backward from stage k-1 to stage 1, compute:

$$\text{cost}(i) = \min \text{ over all edges } (i, j) \text{ of } \{ \text{cost}(i, j) + \text{cost}(j) \}.$$
4. Store the vertex j that gives the minimum cost as the next vertex on the path from i.
5. Starting from the source, follow the stored next-vertex pointers forward to trace the minimum cost path.

Backward Approach:

1. Set $\text{cost}(\text{source}) = 0$.
2. For each vertex j, moving forward from stage 2 to stage k, compute:

$$\text{cost}(j) = \min \text{ over all edges } (i, j) \text{ of } \{ \text{cost}(i) + \text{cost}(i, j) \}.$$
3. Store the vertex i that gives the minimum cost as the predecessor of j on the path.
4. Starting from the destination, follow the stored predecessor pointers backward, then reverse the sequence to get the minimum cost path from source to destination.

Program

// write a program to implement multistage graph problem using dynamic programming and find the minimum cost from the source vertex to the destination vertex. using forward and backward approach.

```
#include <stdio.h>
#include <limits.h>

#define N 12          // number of vertices (0-indexed)
#define INF INT_MAX

int cost[N][N];        // cost[i][j] = cost of edge i -> j, INF if no edge
int stage[N];          // stage number of each vertex

void forwardApproach(int n) {
    int dist[N], path[N];

    for (int i = 0; i < n; i++)
        dist[i] = INF;
    dist[n - 1] = 0;      // destination

    for (int i = n - 2; i >= 0; i--) {
        int minCost = INF, minVertex = -1;
        for (int j = i + 1; j < n; j++) {
            if (cost[i][j] != INF && dist[j] != INF && cost[i][j] + dist[j] < minCost)
```

```

{
    minCost = cost[i][j] + dist[j];
    minVertex = j;
}
}
dist[i] = minCost;
path[i] = minVertex;
}

printf("\n--- Forward Approach ---\n");
printf("Minimum cost from source to destination = %d\n", dist[0]);
printf("Path: %d", 0);
int v = 0;
while (v != n - 1) {
    v = path[v];
    printf(" -> %d", v);
}
printf("\n");
}

void backwardApproach(int n) {
    int dist[N], path[N];

    for (int i = 0; i < n; i++)
        dist[i] = INF;
    dist[0] = 0;          // source

    for (int j = 1; j < n; j++) {
        int minCost = INF, minVertex = -1;
        for (int i = 0; i < j; i++) {
            if (cost[i][j] != INF && dist[i] != INF && dist[i] + cost[i][j] < minCost)
            {
                minCost = dist[i] + cost[i][j];
                minVertex = i;
            }
        }
        dist[j] = minCost;
        path[j] = minVertex;
    }

    printf("\n--- Backward Approach ---\n");
    printf("Minimum cost from source to destination = %d\n", dist[n - 1]);

    int revPath[N], count = 0;
    int v = n - 1;
    while (v != 0) {
        revPath[count++] = v;
        v = path[v];
    }
    revPath[count++] = 0;

    printf("Path: ");
    for (int i = count - 1; i >= 0; i--) {

```

```

        printf("%d", revPath[i]);
        if (i != 0) printf(" -> ");
    }
    printf("\n");
}

int main() {
    int n = N;

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            cost[i][j] = INF;

    // Classic 5-stage, 12-vertex multistage graph (vertices 0..11)
    cost[0][1] = 9; cost[0][2] = 7; cost[0][3] = 3;
    cost[1][4] = 4; cost[1][5] = 2; cost[1][6] = 1;
    cost[2][4] = 2; cost[2][5] = 7; cost[2][6] = 5;
    cost[3][4] = 11; cost[3][5] = 8; cost[3][6] = 10;
    cost[4][7] = 6; cost[4][8] = 5;
    cost[5][7] = 4; cost[5][8] = 3;
    cost[6][7] = 5; cost[6][8] = 6;
    cost[7][9] = 4; cost[8][9] = 3;
    cost[7][10] = 2; cost[8][10] = 5;
    cost[9][11] = 3;
    cost[10][11] = 4;

    printf("Multistage Graph - Minimum Cost Path (Dynamic Programming)\n");
    printf("Source = 0, Destination = %d\n", n - 1);

    forwardApproach(n);
    backwardApproach(n);

    return 0;
}

```

Output

Multistage Graph - Minimum Cost Path (Dynamic Programming)
 Source = 0, Destination = 11

--- Forward Approach ---

Minimum cost from source to destination = 20
 Path: 0 -> 1 -> 5 -> 8 -> 9 -> 11

--- Backward Approach ---

Minimum cost from source to destination = 20
 Path: 0 -> 2 -> 4 -> 8 -> 9 -> 11

Conclusion

The minimum cost path from the source vertex to the destination vertex in the multistage graph was successfully computed using dynamic programming. Both the forward and backward approaches produced the same minimum cost, verifying the correctness of the implementation.